AI Authorship of Facetwork Workflows and Handlers

AI Authorship of Facetwork Workflows and Handlers

Companion addendum to the Facetwork thesis (`thesis.md`).

This document summarises an extended discussion between the thesis supervisor and the candidate on the implications of AI agents — rather than human programmers — becoming the primary authors of Facetwork workflows and handlers. It reframes certain thesis arguments, resists the naive forms of some popular extensions, and sets out a constructive set of design proposals that the final thesis forward-looking section (§15.3) summarises and refers back to.

The discussion developed as a sequence of successive refinements. Each section below captures one step of the argument.

---

1. Determinism, reconsidered

A common pitch for workflow systems in the agentic-AI era is that they are *deterministic*, so that non-deterministic AI agents can run them and produce "consistent behaviour and steps." The pitch is popular because it is superficially correct and rhetorically attractive.

The pitch is a **half-truth delivered as a full truth**. There are at least four distinct senses in which a workflow system can be called deterministic:

1. **Deterministic replay** (Temporal, Cadence). The workflow code reproduces the same decisions given the same event history. This is Facetwork's *opposite*: my thesis argues against imposing this constraint.
2. **Deterministic topology**. The workflow graph is fixed at compile time. Given the same inputs, the same sequence of steps runs with the same typed payloads and the same dependency ordering. Facetwork does have this.
3. **Deterministic orchestration**. Step lifecycle, retry semantics, recovery actions, and observability behave predictably across runs. Facetwork has this.
4. **Deterministic end-to-end behaviour**. Same inputs produce the same outputs across runs. Facetwork does *not* have this when handlers are non-deterministic, and LLM-backed handlers are paradigmatically non-deterministic.

Saying "Facetwork is deterministic" without specifying which sense is misleading. Operators who hear the phrase reasonably infer sense (1) or sense (4); neither holds. The argument worth making is stricter and narrower.

2. The correct argument: procedural consistency

The honest version of the argument is **procedural consistency across heterogeneous executors**, not determinism.

If ten different AI agents (backed by different LLMs, different versions, different frameworks) are each asked to "transform this OSM data," they decompose the task differently, invoke different tools in different orders, and produce different intermediate states. If instead those ten agents are each asked to execute a specific FFL workflow, they all follow the same DAG: same steps, same typed inputs and outputs between steps, same dependency ordering, same retry semantics, same observability. Only the leaf computations — what each LLM actually produces inside a given handler — differ.

This buys several concrete properties the bare-agent approach lacks:

- **Auditability of method, not just output.** The FFL source *is* the procedure. Regulated domains can audit the procedure even if the leaf outputs vary.
- **Meaningful benchmarking across LLMs.** Comparing two LLMs at a fixed facet holds everything else equal. Comparing them at "do this task however you want" compares incomparable things.
- **Fix-once-apply-everywhere.** Fixing a flaky step fixes it for every consumer of the workflow. In an agent-decomposes-the-task world, each agent has its own "step 5" that is differently flaky.
- **Interchangeable implementations.** A facet's typed contract is stable; the handler behind it can be an LLM today, a deterministic algorithm tomorrow, a different LLM next year. Consumers are unaffected.

A better phrase for what workflow systems offer agentic systems is "**procedural reproducibility with leaf-level variability.**" Honest about what the scaffold buys; honest about what it does not.

3. The shift in authorship

A natural observation follows: in practice, the thesis's work on Facetwork — workflows and handlers — has been produced largely by an AI agent, with the human supervisor acting as director and reviewer. This suggests a shift in the authorship model the thesis describes.

The thesis argues that FFL separates *domain programmers* (who write workflows) from *service-provider programmers* (who write handlers). The extended argument is that both roles can be filled by AI agents, with humans serving as:

- **Directors** who specify intent in natural language.
- **Reviewers** who approve generated artifacts.
- **Curators** of the library of approved artifacts.

This is higher-leverage than authorship: one reviewer can approve what would have taken ten authors to write. The review is tractable because FFL was designed to be readable by domain experts, which happens also to make it readable by reviewers of AI-generated workflows, and because the FFL compiler catches the boring class of errors automatically.

4. Sealed skills

Once generated, a workflow can be **sealed** — frozen into a reusable, inspectable, versioned artifact. The concept parallels Docker images, WASM modules, and signed smart contracts, adapted to workflows.

A sealed skill is, concretely:

- An FFL source file (the workflow definition).
- Pinned handler registrations (specific versions of specific implementations, addressed by content hash).
- A declared input/output contract.
- Provenance metadata: which model generated it, which prompt, which reviewer approved it, which tests passed at seal time.
- A signature from the reviewer or sealer.
- A stable content-addressable identifier.

Sealing makes the artifact immutable; new versions are new artifacts, not mutations of the old. Any Facetwork runtime that can parse FFL and resolve the named handler registrations can execute a sealed skill. The operational model becomes: **AI generates, human reviews, skill seals, fleet executes** — a clean division of labour that falls out of the FFL/handler separation almost for free.

5. Two arguments worth resisting

Two plausible-sounding extensions of the AI-authorship premise deserve explicit pushback, because both get the tradeoffs wrong.

5.1 "Humans need not read the workflow language"

The strong form of the argument: since AI reads and writes the workflow language, humans only need a human-readable description of what was generated. The language can be optimised for AI — dense, binary, whatever fits the model — with no concern for human readability.

The flaw. The machine-readable form is authoritative; the description is an approximation. Natural language cannot distinguish between all pairs of subtly different workflows. When the description diverges from the artifact — and it will — the executed behaviour is determined by the artifact, not by the description.

Concrete risks:

1. **Debugging collapses.** A 3 AM production incident needs someone who can read the artifact. "Ask the AI" is a reasonable first step; "read the code" must remain a fallback.
2. **Audit fails as a matter of policy.** Regulated domains require that a qualified human can read what was executed. No regulator will accept "the AI told us it does X" as evidence.
3. **Ontological lock-in.** If only AIs can read the language, humans are dependent on whichever AI can read it. That AI becomes irreplaceable infrastructure. If it becomes unavailable, expensive, or compromised, the repository of artifacts is uninspectable.
4. **The cost of readability is near zero.** FFL is small, declarative, and domain-readable by design. The argument "optimise for AI" would have force if human-readability imposed a significant tax. It does not.

The correct line. *Most* humans *most* of the time do not need to read FFL; a domain expert's day-to-day interaction can be through the description. What must remain true is that *some* humans in *some* roles — compliance reviewers, senior engineers, incident responders, auditors — retain the ability to read the artifact when they need to. The population shrinks; it does not go to zero. Keep both: FFL stays the source of truth and stays human-readable; the description is a generated convenience layer.

5.2 "FFL can be more expressive now that AI writes it"

The plausible version of the argument: FFL was kept simple because non-programmers were meant to author it. With AI authoring, that constraint is gone, so the language can be made more expressive.

The flaw. This conflates two senses of "simple." FFL's simplicity is not primarily about author ergonomics; it is about what the *compiler, runtime, and tooling* can do with the program. The restrictions — bounded iteration, no user-defined generics, no general recursion, statically resolvable references — exist because they make the compiler able to type-check every reference, the runtime able to analyse topology statically, the dashboard able to render execution graphs, the recovery actions able to operate at step granularity, and the block evaluator able to guarantee progress without locks.

Those properties are independent of *who* authors the language. They are what give the DSL its value over code-as-workflow in the first place. Relaxing the restrictions because the author changed loses those properties and degrades the DSL into something closer to Temporal's workflow-as-code, which the thesis explicitly argues against.

The counter-intuitive direction. AI authorship argues for *more* constraint, not less. The research literature on languages designed for program synthesis (Dafny, CVC5's SyGuS, Rosette, Sketch, Lean tactic languages) consistently makes its target languages more restrictive, because synthesisers work better with smaller search spaces and stronger type information. An LLM generating code is, at a high level, a synthesiser with a language-model prior. Giving it a richer language gives it more ways to go wrong and the compiler fewer ways to catch mistakes.

What is worth adding. Specific features — sum types, refinement types, structural row polymorphism, better composition primitives — are worth considering on their own merits because they add expressiveness *without* sacrificing static checkability. What is not worth doing is lifting restrictions (recursion, unbounded dynamic graphs, arbitrary control flow) on the grounds that AI can handle the complexity. The compiler cannot.

6. Concrete design proposals

With the false paths ruled out, the constructive direction follows. Given that AI agents are the primary authors and humans are reviewers and operators, the following specific changes serve the resulting system best. None relax the static discipline; most strengthen it.

6.1 Strengthen the compile-time contract

- **Effect annotations** on every facet: `@pure`, `@idempotent`, `@network`, `@llm`, `@disk-write`, `@external-side-effect`. AI can reason about them; reviewers read them at a glance; the runtime picks retry and sealing policy from them. An `@external-side-effect` handler not also marked `@idempotent` refuses to be the target of `Re-run From Here` without explicit operator confirmation.
- **Refinement types and sum types.** Return fields typed as `Int where x >= 0 and x <= 100` rather than bare `Int`. Sum types (`Result = Ok(T) | Err(E)`) replace loose `error: dict | None` conventions. AI generates these more precisely than humans do, and the compiler catches more.
- **Pre- and post-conditions on facet signatures.** Lightweight contracts: `requires x > 0`, `ensures result.count >= 0`. A small theorem prover or SMT backend at compile time catches whole classes of AI hallucination before execution.

6.2 Make generation itself a first-class concern

- **Schema inference from examples.** Instead of the AI hallucinating a schema, the author provides two or three concrete example payloads; the compiler infers the most specific schema that accepts them and rejects generated workflows that then violate it. Closes the "guessed at the shape" failure mode.
- **Two-phase generation.** First pass: the AI generates the FFL and a stub handler whose only job is to typecheck and return a valid example output. The compiler verifies the stub. Second pass: the AI generates the real implementation against the verified contract. Two-phase generation turns out to be substantially more reliable than single-phase.
- **Structured docstring annotations.** `@purpose`, `@inputs`, `@outputs`, `@failure-modes`, `@idempotent` — filled in by the AI, cross-checked against the code by the compiler, rendered in the dashboard for reviewers. Not free text. The AI's explanation of its own output becomes part of the checked artifact.

6.3 First-class sealed-skill infrastructure

- **Content-addressed skill registry.** A sealed skill is `FFL source + pinned handler registrations + contract + provenance`, hashed into a stable identifier. Immutable. Versioned. Signed by the reviewer.
- **Skill discovery protocol** (MCP-style, extending the existing MCP server). Agents query: "find a sealed skill whose contract is `(PBF) -> GeoJSON`" and receive existing skills before re-generating. Essential for compounding value; without discovery, every team regenerates the same workflows from scratch.
- **Regeneration as semantic diff, not replacement.** When a skill is regenerated from the same specification, the system presents a semantic diff — "added a `catch` on step 3," "tightened the return type of `Sum` from `Int` to `Int where x > 0`." Reviewers approve at the semantic level rather than the syntactic one.

6.4 Review and testing tooling

- **Auto-generated property-based tests.** The AI generates a workflow; the system generates inputs that exercise each branch, runs them in a sandbox, and reports coverage. Sealing requires the suite to pass.
- **Confidence markers on generated output.** The AI annotates its output with per-section confidence; reviewers focus attention on low-confidence sections. Markers are logged with the sealing metadata so that post-hoc analysis can study correlation between AI confidence and actual correctness.
- **Counterfactual replay.** "What would this workflow have produced if step 5 had used a different LLM?" — re-run downstream from step 5 with alternative handlers. Useful for incident triage and for benchmarking LLMs within a fixed procedural scaffold (§2).

6.5 What to remove

- **Retire the `script python` escape hatch.** It exists to let humans reach for a general-purpose language when FFL does not cover something. AI agents should instead generate a new typed facet and its handler; if they cannot, that is a signal to improve FFL, not a reason to keep the escape hatch. Removing it closes a common backdoor for un-checkable logic.
- **Retire any syntax that does not statically check.** Every FFL feature should either be checkable by the compiler or flagged as an explicit dynamic surface. The dynamic surface should shrink over time.

6.6 What to preserve

- **The grammar stays small.** Restrictions (bounded iteration, no general recursion) stay. The arguments from §5.2 apply doubly in an AI-generation world.
- **The FFL/handler separation stays.** Generation is a two-phase process whose phases have different properties; conflating them sacrifices the static checkability that makes AI generation tractable.
- **Human-readability stays.** For the reasons in §5.1, the artifact remains inspectable. Review is not a matter of trusting AI self-reports.

7. Summary

The shift from human-authored to AI-authored Facetwork is not a departure from the thesis's design position; it is an evolution that *vindicates* that position. The DSL's smallness and type discipline, defended in the thesis as supporting domain-expert authorship, turn out to support AI authorship for the same underlying reasons: the compiler catches boring mistakes, the runtime catches operational mistakes, and the review burden is bounded by the size of the semantic surface.

The false paths — optimising the language for AI at the cost of human inspectability, or relaxing restrictions because "AI can handle complexity" — are tempting and wrong for specific, identifiable reasons.

The correct direction is a set of additive refinements: richer *checkable* types, effect annotations, lightweight contracts, two-phase generation with typed stubs, structured provenance, a content-addressable skill registry, a discovery protocol, semantic-diff review, property-based testing, confidence markers, counterfactual replay. Removal of the `script python` escape hatch and of any un-checkable surface. Preservation of everything that makes the compiler effective.

The operational shape that results is: **AI generates, human reviews, skill seals, fleet executes**. The thesis's four design properties — typed DSL, lock-free coordination, state-persisted recovery, live updatability — are unchanged; a fifth property, *AI-author-ready artifact discipline*, is added. The thesis's forward-looking section (§15.3) flags this direction as the fourth natural evolution of the design, and this document is its extended form.

End of addendum.